

Khant Thu Aung/ 105292912

4.2(Iteration-2)

GameObject.cs

```
namespace SwinAdventure
```

```
{
```

```
    public abstract class GameObject : IdentifiableObject
```

```
    {
```

```
        private string _description;
```

```
        private string _name;
```

```
        public GameObject(string[] ids, string name, string desc) : base(ids)
```

```
        {
```

```
            _name = name;
```

```
            _description = desc;
```

```
        }
```

```
        public string Name
```

```
        {
```

```
            get{return _name;}
```

```
        }
```

```
        public string ShortDescription
```

```
        {
```

```
            get{return _name+" (" + FirstID+ ")";}
```

```
        }
```

```
        public virtual string FullDescription
```

```
        {
```

```
        get{return _description;}
    }
}
}
```

Item.cs

```
namespace SwinAdventure
```

```
{
    public class Item : GameObject
    {
        public Item(string[] idents, string name, string desc): base(idents,name,desc)
        {

        }
    }
}
```

Inventory.cs

```
namespace SwinAdventure
```

```
{
    public class Inventory
    {
        private List<Item> _items;

        public Inventory()
        {
            _items = new List<Item>();
        }
    }
}
```

```
}  
  
public bool HasItem(string id)  
{  
    return _items.Any(_item => _item.AreYou(id));  
}  
  
public void PutItem(Item item)  
{  
    _items.Add(item);  
}  
  
public Item? Take(string id)  
{  
    foreach (var item in _items)  
    {  
        if (item.AreYou(id))  
            return item;  
    }  
    return null;  
}  
  
public Item? Fetch(string id)  
{  
    foreach (Item item in _items)  
    {  
        if (item.AreYou(id)) return item;  
    }  
    return null;  
}
```

```

public string ItemList
{
    get
    {
        string list = "";
        foreach (Item item in _items)
        {
            list += item.ShortDescription + "\n";
        }
        return list.TrimEnd();
    }
}

```

Player.cs

```

using System.Reflection.Metadata.Ecma335;

namespace SwinAdventure
{
    public class Player : GameObject
    {
        private Inventory _inventory;

        public Player(string name, string desc): base( new string[] { "me", "inventory"}, name,
desc)
        {
            _inventory = new Inventory();

```

```

    }

    public GameObject Locate(string id)
    {
        if (AreYou(id)) return this;
        return _inventory.Fetch(id);
    }

    public override string FullDescription
    {
        get
        {
            return $"You are {Name}, {base.FullDescription}.\nYou are
carrying:\n{_inventory.ItemList}";
        }
    }

    public Inventory Inventory
    {
        get { return _inventory; }
    }
}

```

TestItem.cs

```

using NUnit.Framework.Legacy;

using SwinAdventure;

namespace UnitTest
{

```

```

[TestFixture]
public class ItemTests
{
    [Test]
    public void TestItemIsIdentifiable()
    {
        var item = new Item(new string[] { "sword", "blade" }, "bronze sword", "A shiny bronze sword.");

        ClassicAssert.IsTrue(item.AreYou("sword"));
        ClassicAssert.IsTrue(item.AreYou("blade"));
        ClassicAssert.IsFalse(item.AreYou("axe"));
    }

    [Test]
    public void TestShortDescription()
    {
        var item = new Item(new string[] { "sword", "blade" }, "bronze sword", "A shiny bronze sword.");

        ClassicAssert.AreEqual("bronze sword (sword)", item.ShortDescription);
    }

    [Test]
    public void TestFullDescription()
    {
        var item = new Item(new string[] { "sword" }, "bronze sword", "A shiny bronze sword.");

        ClassicAssert.AreEqual("A shiny bronze sword.", item.FullDescription);
    }
}

```

```
}  
}
```

TestInventory.cs

```
using NUnit.Framework.Legacy;
```

```
using SwinAdventure;
```

```
namespace UnitTest
```

```
{
```

```
    [TestFixture]
```

```
    public class InventoryTests
```

```
    {
```

```
        private Inventory inventory;
```

```
        private Item item1;
```

```
        private Item item2;
```

```
        [SetUp]
```

```
        public void Setup()
```

```
        {
```

```
            inventory = new Inventory();
```

```
            item1 = new Item(new string[] { "sword" }, "bronze sword", "A shiny bronze sword.");
```

```
            item2 = new Item(new string[] { "shield" }, "wooden shield", "A sturdy wooden shield.");
```

```
            inventory.PutItem(item1);
```

```
            inventory.PutItem(item2);
```

```
        }
```

```
        [Test]
```

```
public void TestFindItem()
{
    ClassicAssert.AreEqual(item1, inventory.Fetch("sword"));
    ClassicAssert.AreEqual(item2, inventory.Fetch("shield"));
}
```

```
[Test]
public void TestNoItemFind()
{
    ClassicAssert.IsNull(inventory.Fetch("axe"));
}
```

```
[Test]
public void TestFetchItem()
{
    var fetchedItem = inventory.Fetch("sword");
    ClassicAssert.AreEqual(item1, fetchedItem);
    ClassicAssert.IsTrue(inventory.HasItem("sword"));
}
```

```
[Test]
public void TestTakeItem()
{
    var takenItem = inventory.Take("sword");
    ClassicAssert.AreEqual(item1, takenItem);
    ClassicAssert.IsFalse(inventory.HasItem("axe"));
}
```



```

    }

    [Test]
    public void TestItemList()
    {
        string expected = "bronze sword (sword)\nwooden shield (shield)";
        ClassicAssert.AreEqual(expected, inventory.ItemList);
    }
}

```

TestPlayer.cs

```

using NUnit.Framework.Legacy;
using SwinAdventure;
namespace UnitTest
{
    [TestFixture]
    public class PlayerTests
    {
        private Player player;
        private Item sword;
        private Item shield;

        [SetUp]
        public void Setup()
        {
            player = new Player("Hero", "a brave warrior");

```

```
        sword = new Item(new string[] { "sword" , "bronze sword"},"a bronze sword" ,"A shiny  
bronze sword.");
```

```
        shield = new Item(new string[] { "shield" , "wooden shield"},"a wooden shield", "A  
sturdy wooden shield.");
```

```
        player.Inventory.PutItem(sword);  
        player.Inventory.PutItem(shield);  
    }
```

```
[Test]
```

```
public void TestPlayerIsIdentifiable()  
{  
    ClassicAssert.IsTrue(player.AreYou("me"));  
    ClassicAssert.IsTrue(player.AreYou("inventory"));  
    ClassicAssert.IsFalse(player.AreYou("you"));  
}
```

```
[Test]
```

```
public void TestPlayerLocatesItems()  
{  
    ClassicAssert.AreEqual(sword, player.Locate("sword"));  
    ClassicAssert.AreEqual(shield, player.Locate("shield"));  
}
```

```
[Test]
```

```
public void TestPlayerLocatesItself()  
{  
    ClassicAssert.AreEqual(player, player.Locate("me"));  
}
```

```
        ClassicAssert.AreEqual(player, player.Locate("inventory"));
    }
```

```
[Test]
public void TestPlayerLocatesNothing()
{
    ClassicAssert.IsNull(player.Locate("axe"));
}
```

```
[Test]
public void TestPlayerFullDescription()
{
    string expected = "You are Hero, a brave warrior.\nYou are carrying:\n" +
        "a bronze sword (sword)\n" +
        "a wooden shield (shield)";
    ClassicAssert.AreEqual(expected, player.FullDescription);
}
}
}
```

